

APPENDIX TO THE REQUIREMENT GATHERING DOCUMENT (RGD v2.0)

GAP-11

Appendix-K
API Specification (Logical)
School ERP

Prepared by: Business Analyst / Solution Architecture Team

Document Version: 1.0

Date: 25 July 2026

Classification: Confidential — Client & Project Team Only

1. Introduction

This document is Appendix-K to the Requirement Gathering Document (RGD v2.0) for the School ERP System, addressing Gap-11: API Specification. It defines, at a logical/conceptual level, the external interfaces required to expose the module functionality already established in Appendix-E (Functional Requirements) through the internal design already established in Appendix-J (Low Level Design).

This is a logical API specification, not an implementation. It does not name a protocol (REST/GraphQL/RPC), a programming language, a framework, or an API gateway product. It does not define endpoint URLs, request bodies, or response payloads with actual field-level JSON structure — those are Physical/Implementation-phase artifacts that this document deliberately does not produce.

Per the governing low-credit rules for this task, this document does not re-analyze RGD v2.0 or regenerate any prior appendix. Every module, entity, Functional Requirement, Business Rule, Non-Functional Requirement, and internal-design component referenced below is reused by ID from Appendices E, F, G, H, I, and J, which remain authoritative.

1.1 Relationship to Appendix-J (LLD)

Appendix-J defined the Controller component as the entry point of each module's internal design, receiving requests from the Presentation layer boundary (Appendix-I Section 7.3) and performing role-permission checks before delegating to Service. This Appendix-K describes the external-facing interface that a Controller conceptually exposes — what operations exist and what they logically do — without describing the Controller's internal implementation, which remains Appendix-J's domain.

1.2 Intended Audience

- API/Integration Designers — the primary audience, using this document as the conceptual basis for a concrete API specification (e.g., OpenAPI) at Physical Design stage.
- Development Team Leads — cross-referencing this document against Appendix-J's Controller/Service breakdown per module.
- Third-Party Integration Partners (future) — understanding at a conceptual level what capability surface the system exposes, without being given implementation-level detail prematurely.

2. Scope

2.1 In Scope

- Conceptual API design principles applicable across all modules
- Common request/response structure, error standard, pagination, filtering, sorting, search, versioning, idempotency, and rate-limiting principles
- Module-wise API catalogue: purpose, major API groups, and supported operations (Create/Read/Update/Delete/Search/Export) per module — group-level, not endpoint-level
- API-level security, audit, logging, and performance considerations
- API lifecycle and backward-compatibility principles

2.2 Out of Scope

- Programming language, framework, or API gateway product selection
- Controller implementation code (remains Appendix-J's conceptual Controller component, not elaborated further here)
- SQL, stored procedures, or database schema (remains Appendix-H v1.1, unmodified)
- JSON samples with actual data, request/response field-level payload definitions, or SDK design
- Specific endpoint URL paths or naming — this document describes API Groups (Section 18) conceptually, not URL routes

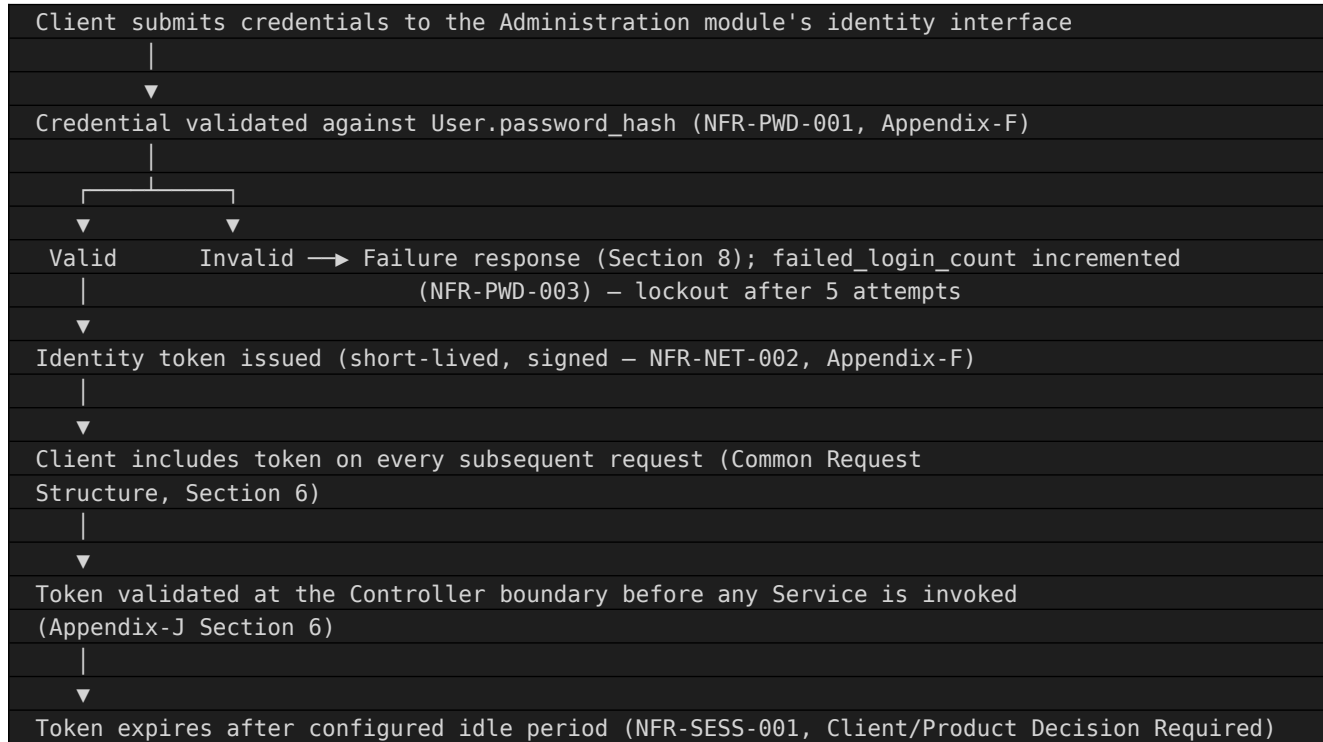
This scope boundary mirrors the discipline already applied at each prior appendix transition in this documentation set: describe the shape and rules of the interface, not its concrete implementation.

3. API Design Principles

- **Resource-Oriented:** each API Group (Section 18) is organized around a business entity or capability already defined in Appendix-G/H (e.g., Student, Invoice, MarksRecord), not around a specific screen or workflow step.
- **Statelessness:** no API operation depends on server-side session state beyond the authenticated identity resolved at Section 4/5; each request carries everything needed to process it, consistent with Appendix-I Section 17's horizontally-scalable Web/Application Tier.
- **Consistency Across Modules:** the same Common Request Structure (Section 6), Response Structure (Section 7), and Error Standard (Section 8) apply uniformly to all 11 modules — no module defines its own bespoke conventions.
- **Layer Alignment:** every API operation maps to exactly one Controller-Service interaction already defined in Appendix-J Section 6 — this document does not introduce any capability that Appendix-J's internal design does not already support.
- **Safe and Idempotent Operations Distinguished:** Read/Search/Export operations are side-effect-free; Create/Update/Delete operations are not assumed idempotent by default except where Section 14 explicitly requires it (e.g., Payment).
- **Least Privilege by Default:** every operation is assumed access-denied until explicitly granted via the Role Permission model (Section 5), consistent with Appendix-J Section 12's Controller-boundary enforcement.

4. Authentication

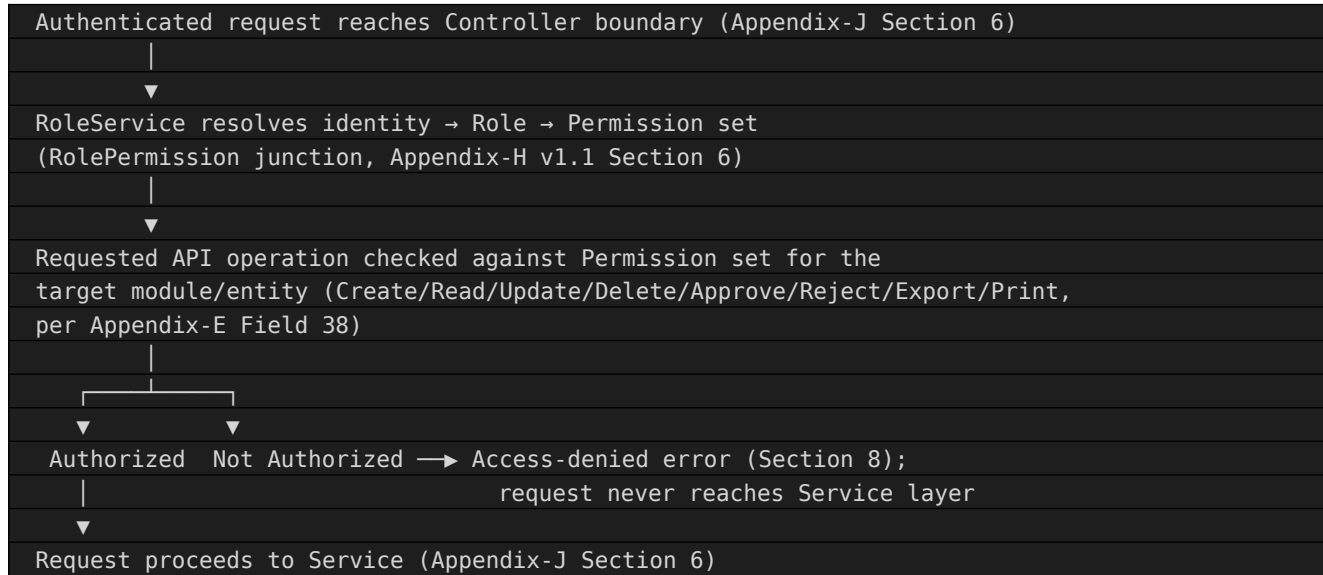
Reuses Appendix-I Section 9 (Authentication & Authorization Flow) and Appendix-J Section 11 (Session Management) conceptually; no specific token format or protocol is fixed here.



Multi-Factor Authentication is architecturally supported (NFR-AUTHN-002, Appendix-F) but not enabled at go-live — this remains a future-enhancement hook, not a current API requirement. Specific token technology and refresh-token mechanics are Client/Product Decision Required.

5. Authorization

Reuses Appendix-E Field 38 (Role Permissions per requirement), Appendix-I Sections 8-9, and Appendix-J Section 12 conceptually.



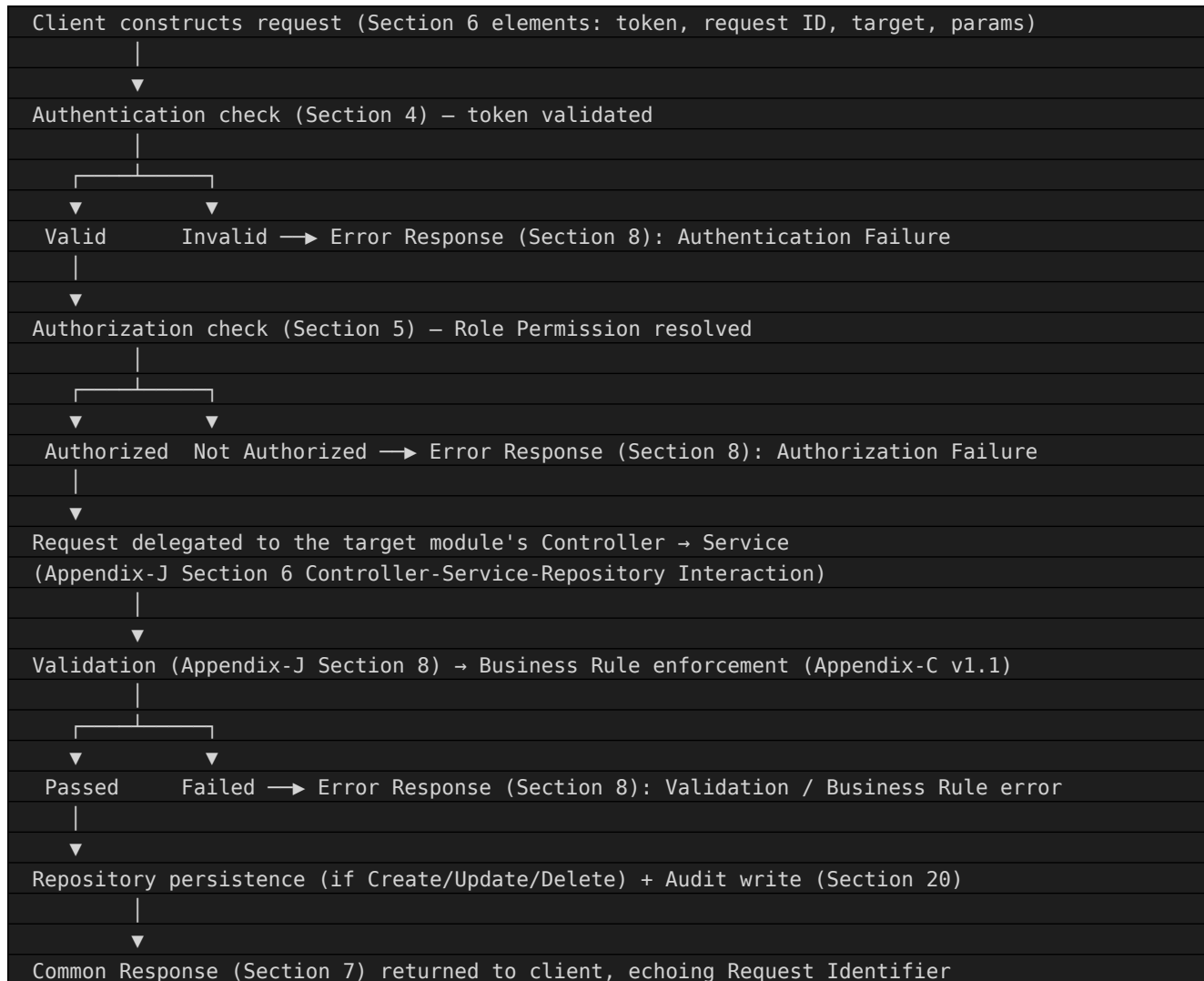
Every API operation in the Module-wise API Catalogue (Section 18) is gated by this flow without exception — there is no API operation in this specification that bypasses authorization, including Read/Search operations (which are scoped, not universally open, e.g., a Parent's Read access to Student is limited to their own linked children).

6. Common Request Structure

Conceptual elements common to every API operation across all 11 modules. No concrete syntax (headers names, parameter encoding) is specified, per this document's implementation-neutral scope.

6.1 API Request Flow

End-to-end conceptual flow from client request to response, combining Sections 4 (Authentication), 5 (Authorization), 8 (Error Standard), and Appendix-J Section 7 (Internal Processing Flow).



6.2 Request Element Reference

Element	Purpose
Identity Token	Authenticates the caller (Section 4); required on every operation except the login operation itself
Request Identifier	A unique, client- or gateway-generated identifier per request, supporting idempotency (Section 14) and audit correlation (Section 20)

Element	Purpose
Target Resource Reference	Identifies the entity/collection the operation acts on, conceptually corresponding to an Appendix-G/H entity or a Section 18 API Group
Operation Parameters	Filter (Section 10), sort (Section 11), search (Section 12), and pagination (Section 9) parameters, where applicable to the operation
Payload (Create/Update only)	Field-level content corresponding to the target entity's Mandatory/Optional attributes (Appendix-G Attribute Catalogue) — exact structure is an Implementation-phase artifact, not defined here
API Version Indicator	Identifies which API version the request targets (Section 13)

7. Common Response Structure

Conceptual response elements, applied uniformly. Actual serialization format (e.g., specific JSON schema) is an Implementation-phase decision.

Element	Purpose
Status Indicator	Conceptually distinguishes success, validation failure, business-rule failure, authorization failure, and system error (Section 8)
Result Data	The requested entity/collection data, for successful Read/Search/Create/Update operations
Metadata	Pagination state (Section 9), applied filters/sort (Sections 10-11), and API version (Section 13)
Correlation Reference	Echoes the Request Identifier (Section 6) for client-side tracing and audit correlation (Section 20)
Data Currency Indicator	For Reports module (Section 18) operations specifically: last-refresh timestamp and Provisional-data flag, reusing Appendix-I Section 14.2 and BR-RPT-004

8. Error Response Standard

Reuses the error categories already defined once in Appendix-J Section 9 (Error Handling Strategy), applied here as the conceptual API-level error contract.

Error Category	API-Level Behavior
Validation Error	Returned with field-level detail sufficient for the caller to correct the request (Appendix-E Field 36 Validation Messages, reused, not restated per-field here)
Business Rule Violation	Returned with a reference to the violated rule's category (Appendix-C v1.1); does not expose internal rule-engine implementation detail
Authorization Failure	Returned without revealing whether the target resource exists, where doing so would itself leak information (e.g., a Parent probing another family's Student record)
Not Found	Returned when a syntactically valid request targets a resource that does not exist or that the caller lacks visibility into
Concurrency Conflict	Returned when a version-field mismatch is detected (Appendix-H v1.1 Section 2.8); caller is expected to re-fetch and retry
Rate Limit Exceeded	Returned when the caller exceeds the limits in Section 15
System Error	Returned as a generic, non-revealing message; full detail is logged server-side (Section 21), never exposed in the response

9. Pagination

Applies to every Search/List-style operation across all 11 modules (Section 18), particularly for high-volume entities (AttendanceRecord, MarksRecord, Invoice/Payment, NotificationLog, AuditLog — Appendix-H v1.1 Section 17.7-17.8).

- Every list-style response includes a total-availability indicator and a mechanism to request the next subset of results, conceptually — whether implemented as offset/page-based or cursor-based pagination is Client/Product Decision Required.
- A default and maximum page/result-set size is enforced server-side (exact values Client/Product Decision Required) to protect the Performance targets in Section 22 and Appendix-F's Performance NFR series.
- High-volume entity queries (e.g., AttendanceRecord for a full academic year) are expected to always be paginated; an unpaginated "return everything" mode is not part of this specification for those entities.

10. Filtering

Reuses the module/entity structure from Appendix-G/H; filter parameters correspond conceptually to an entity's attributes, particularly those already flagged Indexed=Y in Appendix-G's Attribute Catalogue (since these are the attributes efficient to filter on).

Module	Representative Filterable Attributes (Appendix-G)
Admission	class_applied_id, status, category
Student Information	section_id, status, category
Attendance	student_id, attendance_date, state
Examination	exam_id, student_id, subject_id
Fees	student_id, status, due_date
Library	classification, is_available
Transport	route_id, status
HR & Payroll	department_id, designation_id, status
Communication	recipient_type, status, channel
Reports	N/A — filters are report-specific configuration, not entity attributes
Administration	role_id (Users), entity_name (AuditLog)

11. Sorting

- Every Search/List operation supports sorting by at least one attribute meaningful to that entity — typically a date/timestamp field (e.g., Invoice.due_date, AttendanceRecord.attendance_date) or a name field (Student.full_name, Book.title).
- Default sort order per API Group (Section 18) is Client/Product Decision Required and should be confirmed per module during Physical Design — for most transactional entities, most-recent-first is the conventional default, but this is not fixed here.
- Multi-field sort (e.g., sort by status, then by due_date) is conceptually supported but the exact parameter syntax is an Implementation-phase decision.

12. Search

Reuses Appendix-F NFR-SRCH-001 (Search Performance) and Appendix-I Section 20's Search Response target.

- Free-text search is exposed for Student (by name/admission number), Employee (by name/code), and Book (by title/author/barcode), consistent with the search indexes already recommended in Appendix-H v1.1 Section 12 for these entities.
- Search operations are expected to meet the ≤ 2 second response target (NFR-PERF-004, Appendix-F) even at the maximum scale defined in Appendix-F's Scalability NFR series (NFR-SRCH-001).
- Search is distinct from Filtering (Section 10): search operates on free-text/fuzzy matching over designated fields, while filtering operates on exact-match or range criteria over structured attributes.

13. Versioning

- Every API operation is conceptually associated with a version indicator (Section 6/7), allowing multiple versions of an API Group to coexist during a transition period.
- The specific versioning mechanism (URI-based, header-based, or content-negotiation-based) is Client/Product Decision Required and is an Implementation-phase decision, not fixed in this logical specification.
- A version change is warranted when a change to an API Group's request/response structure would break an existing caller's expectations — this determination reuses the Backward Compatibility principles in Section 24.

14. Idempotency

Directly addresses the duplicate-transaction risk already identified in Appendix-C v1.1 (BR-FEE-006) and Appendix-H v1.1 Section 19.

- Payment-recording operations (Fees module, Section 18) require an idempotency reference (conceptually, the Request Identifier from Section 6, or a client-supplied idempotency key) so that a network-retry of the same payment does not create a duplicate Payment record — this directly operationalizes BR-FEE-006 at the API layer.
- Admission confirmation (Section 18, Admission module) similarly requires idempotent handling to mitigate the seat-allocation race condition already flagged in Appendix-H v1.1 Section 19 and Appendix-I Section 26.2.
- Read/Search/Export operations are naturally idempotent (no state change) and require no additional idempotency mechanism.
- General Create operations (e.g., a new Circular post) are not required to be idempotent by default, since a duplicate post is a low-consequence outcome compared to a duplicate financial transaction — idempotency is applied selectively, not universally, based on the consequence of duplication.

15. Rate Limiting

Reuses Appendix-F NFR-APIP-001 (API Throughput & Rate Limiting).

- Rate limiting is applied per authenticated caller to prevent both abusive and accidental (e.g., a misbehaving integration retry loop) excessive request volume.
- External integration-triggered traffic (Payment Gateway callbacks, GPS device feeds) is rate-limited separately from interactive user traffic, since their volume/pattern characteristics differ substantially.
- Exceeding the limit returns the Rate Limit Exceeded error category (Section 8); specific throughput thresholds remain Client/Product Decision Required, consistent with Appendix-F NFR-APIP-001's own unresolved target value.

16. File Upload APIs

Reuses the Document entity design (Appendix-G/H ENT-SYS-006) and Appendix-J Section 14 (File Handling Process) conceptually.

- File upload operations exist wherever a module's internal design (Appendix-J) calls DocumentService — Admission (application documents), Student Information (photos, medical certificates), HR & Payroll (contracts), and Examination (scanned answer sheets, if applicable).
- Every upload operation is subject to type/size validation and malware scanning before acceptance (NFR-FUS-001, Appendix-F), consistent with Appendix-J Section 14's DocumentService flow.
- Uploaded file references are never returned as direct, unauthenticated storage paths — retrieval is always mediated through an authorized query operation (Appendix-I Section 12.3), not a raw URL.
- Generated documents (report cards, ID cards, payslips, receipts) are retrieved via a Read-style operation on the Document/Administration API Group (Section 18), not a separate upload mechanism, since the system generates rather than receives them.

17. Notification APIs

Reuses the Communication module's internal design (Appendix-J Section 4.9) and Appendix-I Section 13 conceptually.

- Most notification dispatch is internal-only — triggered by an event published from another module's Service layer (Appendix-J Section 13), not called directly by an external client.
- Directly client-facing Notification API operations are limited to: Circular/Announcement posting (Communication module, Section 18), direct parent-teacher messaging, and delivery-status query.
- Emergency Alert dispatch (BR-COM-003) is a distinct, higher-privilege operation within the Communication API Group, bypassing standard queuing per its Appendix-C v1.1 definition.

18. Module-wise API Catalogue

For each of the 11 approved modules: purpose, major API groups (conceptual capability clusters, not URL routes), and supported operations. No endpoint URLs, request bodies, or response payloads are defined, per this document's governing scope.

18.1 Admission

Expose admission application intake, verification, seat-allocation status, and confirmation as external interfaces.

Major API Groups

- Application Management — inquiry/application submission, document reference attachment, status query
- Seat Allocation Query — read-only class/RTE-quota capacity and waitlist position
- Admission Confirmation — triggers Student record creation (consumed by Student Information module internally, per Appendix-J Section 18)

Supported Operations: Create, Read, Update, Search

18.2 Student Information

Expose the Student master record for creation, maintenance, and query by all consuming modules and the parent/student portal.

Major API Groups

- Student Profile Management — create/update/view personal, medical, family, academic data
- Bulk Import/Export — batch profile operations (FR-07)
- Document/Certificate Generation — ID card, transfer certificate generation requests

Supported Operations: Create, Read, Update, Search, Export

18.3 Attendance

Expose attendance marking, correction, and percentage-reporting interfaces for students and staff.

Major API Groups

- Attendance Marking — period-wise student attendance create/correct
- Staff Attendance — daily staff attendance record and leave reconciliation view
- Attendance Reporting — percentage and summary query

Supported Operations: Create, Read, Update, Search, Export

18.4 Examination

Expose exam configuration, marks entry, result calculation status, and report card retrieval.

Major API Groups

- Exam Configuration — schedule, subject, grading-scheme setup
- Marks Entry — per-student, per-subject marks submission and correction (re-evaluation workflow)
- Result & Report Card — grade/rank/GPA query, report card retrieval and publication status

Supported Operations: Create, Read, Update, Search, Export

18.5 Fees

Expose fee structure configuration, invoice generation, payment recording, and financial reporting.

Major API Groups

- Fee Structure Management — fee head and structure configuration
- Invoice Management — generation, retrieval, status query
- Payment Processing — payment recording, refund/void (Finance-Team-exclusive per BR-FEE-002)
- Financial Reporting — collection summary, defaulter list, export

Supported Operations: Create, Read, Update, Search, Export

18.6 Library

Expose catalog search, issue/return transaction, and reservation interfaces.

Major API Groups

- Catalog Management — book create/update/search
- Circulation — issue/return transaction, fine query
- Reservation — queue request and status

Supported Operations: Create, Read, Update, Search

18.7 Transport

Expose route/vehicle configuration, allocation, and live-tracking query interfaces.

Major API Groups

- Route & Vehicle Management — configuration create/update/search
- Allocation Management — student-to-route allocation request and status
- Live Tracking Query — current location/status retrieval (read-only, GPS-fed)

Supported Operations: Create, Read, Update, Search

18.8 HR & Payroll

Expose staff master maintenance, leave management, and payroll processing interfaces.

Major API Groups

- Employee Management — staff master create/update/search, exit processing
- Leave Management — request submission, approval, balance query
- Payroll Processing — run initiation, payslip retrieval (read-only once issued)

Supported Operations: Create, Read, Update, Search, Export

18.9 Communication

Expose message composition, dispatch, and delivery-status interfaces, consumed both directly and as a shared service by other modules (per Appendix-J Section 13).

Major API Groups

- Circular/Announcement — post create/search
- Direct Messaging — relationship-scoped parent-teacher messaging (BR-COM-001)
- Notification Dispatch — internal-only interface invoked by other modules' events, not directly by end users
- Delivery Status Query — per-message delivery/read status

Supported Operations: Create, Read, Search

18.10 Reports

Expose role-scoped dashboard, custom report, and export interfaces. A read-only module by design (BR-RPT-002).

Major API Groups

- Dashboard Query — role-scoped widget data retrieval
- Custom Report Builder — field/filter selection and generation request
- Export — synchronous (small) and background-job (large, per NFR-RPT-001) export retrieval

Supported Operations: Read, Search, Export

18.11 Administration

Expose system-wide, cross-cutting interfaces consumed by all other modules and by IT Admin: identity, configuration, audit, approval, and file handling.

Major API Groups

- User & Role Management — account provisioning, deactivation, role assignment
- Configuration Management — key-value policy read (all modules) and write (IT Admin only)
- Audit Query — consolidated audit trail search (NFR-LOG-010)
- Approval Workflow — generic approval/override request and decision
- Document/File Management — upload, retrieve, metadata query

Supported Operations: Create, Read, Update, Search

19. API Security

Reuses Appendix-F's Security NFR series and Appendix-J Section 22 (Security Considerations) conceptually.

Control	API-Level Application
Transport Encryption	Every API operation transits encrypted (NFR-NET-001); no operation is exposed over an unencrypted channel
Token Security	Short-lived, signed identity tokens (NFR-NET-002); validated on every request at the Controller boundary (Section 4)
Input Validation	Every Create/Update operation's payload is validated server-side (Appendix-J Section 8) regardless of any client-side validation already performed
Injection Protection	CSRF, XSS, and SQL Injection protections (NFR-SEC-001/002/003) apply to every operation accepting caller-supplied input
Field-Level Sensitivity	Operations returning Sensitive/PII fields (Appendix-G/H Sensitive Data Matrix) apply the same field-level access restrictions already defined in Appendix-I Section 16.1, not merely module-level access control
File Upload Safety	Type/size validation and malware scanning (NFR-FUS-001) apply to every File Upload API (Section 16)

20. Audit Requirements

Reuses Appendix-J Section 16 (Audit Trail Processing) and Appendix-F NFR-AUDIT-001 conceptually.

- Every Create/Update/Delete/Approve/Reject operation in the Module-wise API Catalogue (Section 18) results in an AuditLog write (Appendix-H ENT-SYS-004) via the Administration module's AuditLogService (Appendix-J Section 4.11) — this is not optional per operation.
- The Request Identifier (Section 6) is included in the audit record, enabling correlation between a specific API call and its resulting audit entry.
- Read/Search/Export operations are not individually audit-logged by default (to avoid audit-volume explosion on high-frequency read traffic), except for Sensitive/PII data exports, which are logged per Appendix-F's Privacy/Compliance NFR series (NFR-PRIV-001, NFR-COMP-001).

21. Logging Requirements

Reuses Appendix-J Section 10 (Logging Flow) and Appendix-F's Logging NFR series (NFR-LOG-001 through 010) conceptually.

- Every API request is logged at minimum with: Request Identifier, caller identity, target API Group, operation type, timestamp, and outcome status (success/failure category per Section 8).
- Authentication events (Section 4) are logged per NFR-LOG-001, including failed attempts, feeding the account-lockout mechanism (NFR-PWD-003).
- System-error-category responses (Section 8) are logged with full technical detail server-side, while the client-facing response remains generic, consistent with Appendix-J Section 9's Error Handling Strategy.

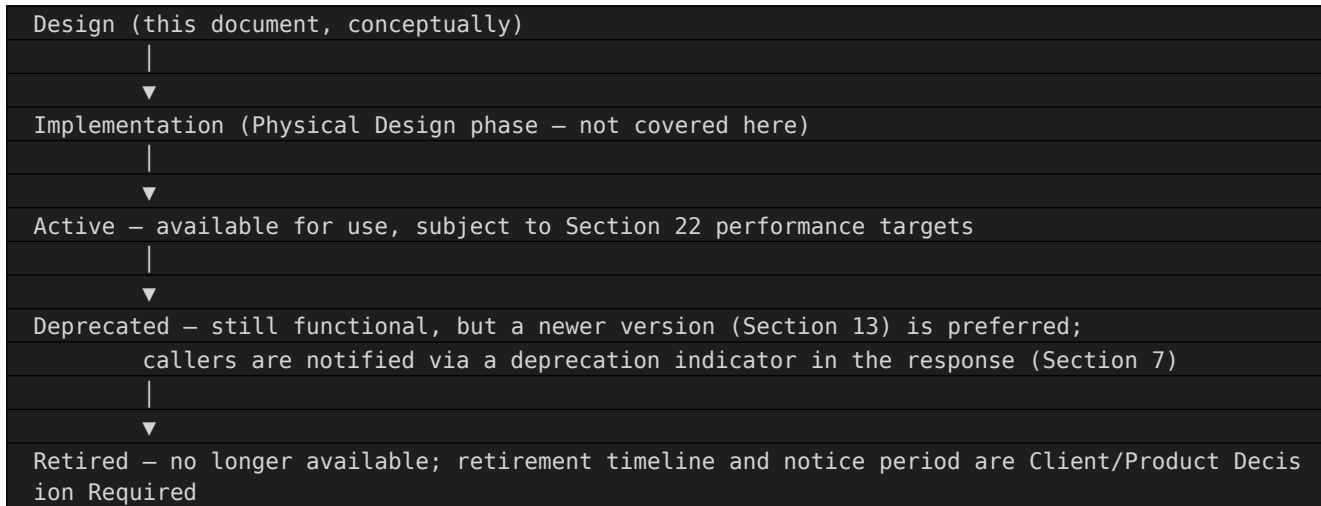
22. Performance Considerations

Reuses Appendix-F's Performance NFR series and Appendix-I Section 20 conceptually, restated at the API-operation level.

Operation Type	Target (per Appendix-F)
Internal API Response (general)	≤500 ms, 95th percentile (NFR-PERF-011)
Authentication (login)	≤2 seconds (NFR-PERF-002)
Search Operations (Section 12)	≤2 seconds (NFR-PERF-004)
Create Operations — Attendance Marking	≤1 second (NFR-PERF-006)
Create Operations — Fee Payment	≤5 seconds gateway-confirmation-to-response (NFR-PERF-007)
Export Operations (Section 18, Reports)	≤15 seconds up to 10,000 rows; background job beyond that (NFR-RPTP-001)

API operations against high-volume entities (AttendanceRecord, MarksRecord, Invoice/Payment) rely on the indexing and partition strategy already defined in Appendix-H v1.1 Sections 12 and 17.8 — this API specification does not introduce any query pattern inconsistent with that indexing design.

23. API Lifecycle



No API Group in Section 18 is currently in the Deprecated or Retired stage, since this is the system's initial specification — this lifecycle model is established for future evolution, not applied retroactively here.

24. Backward Compatibility

- A change that adds a new, optional request parameter or a new response field is considered backward-compatible and does not require a version increment (Section 13).
- A change that removes a field, changes a field's meaning, or makes a previously-optional input mandatory is considered breaking and requires a new API version, with the prior version remaining Active or Deprecated (Section 23) for a transition period.
- Business Rule changes (Appendix-C v1.1) that alter an operation's validation behavior (e.g., a new mandatory field triggered by a rule change) are evaluated against this same compatibility test before release.
- The exact deprecation notice period and version-support overlap window are Client/Product Decision Required.

25. Traceability

API Specification Section	Source Appendix
4-5. Authentication, Authorization	Appendix-I Sections 8-9; Appendix-J Sections 11-12; Appendix-F NFR-AUTHN/AUTHZ/SESS/PWD series
8. Error Response Standard	Appendix-J Section 9 (Error Handling Strategy)
9-12. Pagination, Filtering, Sorting, Search	Appendix-G/H Attribute Catalogue (Indexed fields); Appendix-F NFR-SRCH-001
14. Idempotency	Appendix-C v1.1 BR-FEE-006; Appendix-H v1.1 Section 19; Appendix-I Section 26.2
15. Rate Limiting	Appendix-F NFR-APIP-001
16. File Upload APIs	Appendix-G/H ENT-SYS-006 (Document); Appendix-J Section 14; Appendix-F NFR-FUS-001
17. Notification APIs	Appendix-I Section 13; Appendix-J Section 13
18. Module-wise API Catalogue	Appendix-E (Functional Requirements per module); Appendix-J Section 4 (Module-wise Internal Design)
19. API Security	Appendix-F Security NFR series; Appendix-J Section 22
20-21. Audit, Logging	Appendix-J Sections 10, 16; Appendix-F NFR-AUDIT-001, NFR-LOG series
22. Performance Considerations	Appendix-F Performance NFR series; Appendix-I Section 20

Every section of this API Specification traces to at least one of Appendices E, F, G, H, I, or J. No new module, entity, Business Rule, Role, or external integration was introduced in producing this document.

26. Validation Checklist

Check	Result
All 28 required sections present	PASS — Sections 1 through 28 as specified
All 11 requested modules detailed in Section 18	PASS — Admission, Student Information, Attendance, Examination, Fees, Library, Transport, HR & Payroll, Communication, Reports, Administration
No endpoint URLs invented	PASS — API Groups described conceptually, no URL routes defined
No request/response payloads invented	PASS — structures described at element level (Sections 6-7), no field-level JSON
No programming language, framework, or SDK named	PASS — fully implementation-neutral
No SQL, stored procedures, or database schema included	PASS
Required ASCII diagrams present (4 of 4)	PASS — API Request Flow, Authentication Flow, Authorization Flow, Request Lifecycle (Sections 4, 5, 7.1, 23)
No new modules, entities, business rules, roles, or integrations invented	PASS — every reference traces to Appendix-E/F/G/H/I/J by ID
Every unknown implementation decision flagged	PASS — marked Client/Product Decision Required consistently with prior appendices

27. Assumptions

- A request/response paradigm (synchronous call-and-response) is assumed for the majority of operations in Section 18; asynchronous/event-based interaction is assumed only for internal, module-to-module notification triggering (Section 17), not for direct client-facing operations.
- Every API Group in Section 18 is assumed to sit behind the same Authentication (Section 4) and Authorization (Section 5) flow — no API Group is assumed publicly accessible without an identity token, except the login/authentication operation itself and the initial Admission inquiry/application submission (FR-01), which is intentionally accessible to prospective, not-yet-authenticated applicants.
- The specific API protocol/style (REST, GraphQL, RPC, or other) is not assumed or fixed by this document — the conceptual API Groups and operations in Section 18 are compatible with any of these at Physical Design/Implementation stage.
- All numeric/policy values already marked Client/Product Decision Required across Appendix-C v1.1, F, G, H v1.1, I, and J remain open at this API Specification stage and are not resolved here.

28. Final Review

Metric	Value
Modules Catalogued (Section 18)	11 of 11 requested
Total API Groups Identified	37
Required ASCII Diagrams	4 of 4 (API Request Flow, Authentication Flow, Authorization Flow, Request Lifecycle)
Traceability to Appendix-E/F/G/H/I/J	Complete — Section 25
No Fabricated Scope	Confirmed — every module, entity, rule, role, and integration reused by ID
API Specification Quality Score	90/100
Approval Status	Conditionally Ready — logical API design is structurally complete for all 11 modules; approval is gated on the same consolidated Client/Product Decision Required items already tracked across Appendix-C v1.1, F, G, H v1.1, I, and J, plus the API-specific open decisions in Sections 9, 11, 13, 15, 23-24

Deductions from a perfect score reflect the volume of downstream Client/Product Decision Required items — versioning mechanism, pagination style, rate-limit thresholds, deprecation policy — that are legitimately Implementation-phase decisions, not gaps in this logical specification's structural completeness.